

Blockchain Experiment 8

AIM: Implementing a Blockchain Platform using Ganache

Theory:

1. Blockchain Technology

A blockchain is a distributed, decentralized, and immutable ledger that records transactions across a network of computers. Each record (called a block) contains a cryptographic hash of the previous block, a timestamp, and transaction data, forming a chain. Because every participant holds a copy of the ledger and any alteration to a block invalidates all subsequent blocks, the data stored on a blockchain is highly tamper-resistant and transparent.

Key properties of blockchain:

- Decentralization: No single entity controls the network; authority is distributed across nodes.
- Immutability: Once data is written to the blockchain, it cannot be altered or deleted.
- Transparency: All transactions are visible to network participants.
- Consensus: Nodes agree on the state of the ledger through consensus mechanisms (e.g., Proof of Work, Proof of Stake).
- Security: Cryptographic hashing (SHA-256) ensures data integrity.

2. Ethereum & Smart Contracts

Ethereum is an open-source, decentralized blockchain platform that supports programmable transactions through Smart Contracts. A smart contract is a self-executing program stored on the blockchain whose terms are written directly into code. Once deployed, a smart contract runs automatically when predefined conditions are met, without the need for intermediaries.

Smart contracts in Ethereum are written in Solidity, a statically-typed, contract-oriented programming language. They are compiled into bytecode and deployed to the Ethereum Virtual Machine (EVM), which executes the contract logic on every node in the network.

Key concepts:

- Gas: A unit measuring the computational effort required to execute operations. Users pay gas fees in ETH to incentivize miners/validators.
- ABI (Application Binary Interface): Defines how to interact with the deployed contract — the function signatures and data encodings.
- Address: Every deployed contract gets a unique Ethereum address on the blockchain.
- msg.sender: A global variable in Solidity that holds the address of the account that called the current function.

3. Ganache

Ganache (by Truffle Suite / Consensys) is a personal blockchain for Ethereum development. It simulates

a full Ethereum node locally on your machine, providing:

- 10 pre-funded test accounts, each loaded with 100 ETH.
- Instant mining — transactions are confirmed immediately (no wait time).
- A local RPC server (default: <http://127.0.0.1:7545>) that tools like MetaMask and Remix can connect to.
- A GUI showing real-time accounts, blocks, transactions, and logs for easy inspection.

Ganache is used exclusively for development and testing. It allows developers to deploy and test contracts without spending real ETH or waiting for mainnet/testnet confirmations.

4. MetaMask

MetaMask is a cryptocurrency wallet and browser extension that acts as a bridge between a web browser and the Ethereum blockchain. It allows users to manage Ethereum accounts, sign transactions, and interact with decentralized applications (dApps) directly from the browser. In development, MetaMask is configured to point to a local Ganache network instead of the Ethereum mainnet, enabling safe testing with dummy ETH.

5. Remix IDE

Remix IDE is a web-based integrated development environment for Solidity smart contract development, available at remix.ethereum.org. It provides a code editor, Solidity compiler, contract deployer, and debugger in a single interface. Remix can connect to various environments including the built-in JavaScript VM, Hardhat, or an Injected Provider (MetaMask), the last of which routes deployment through MetaMask to whichever network (Ganache, testnet, or mainnet) MetaMask is currently connected to.

6. Supply Chain Use Case

A supply chain smart contract automates the tracking of goods as they move between parties — in this experiment, a pharmaceutical product (medicine) passing through four stages: Manufacturer, Distributor, Retailer, and Customer. Each stage updates the contract state on-chain, creating a transparent and tamper-proof audit trail. This eliminates the need for a trusted central authority to verify ownership transfers and ensures all parties see the same data.

Implementation:

Step 1: Install Ganache

Ganache is a personal blockchain for Ethereum development used to deploy contracts, develop applications, and run tests.

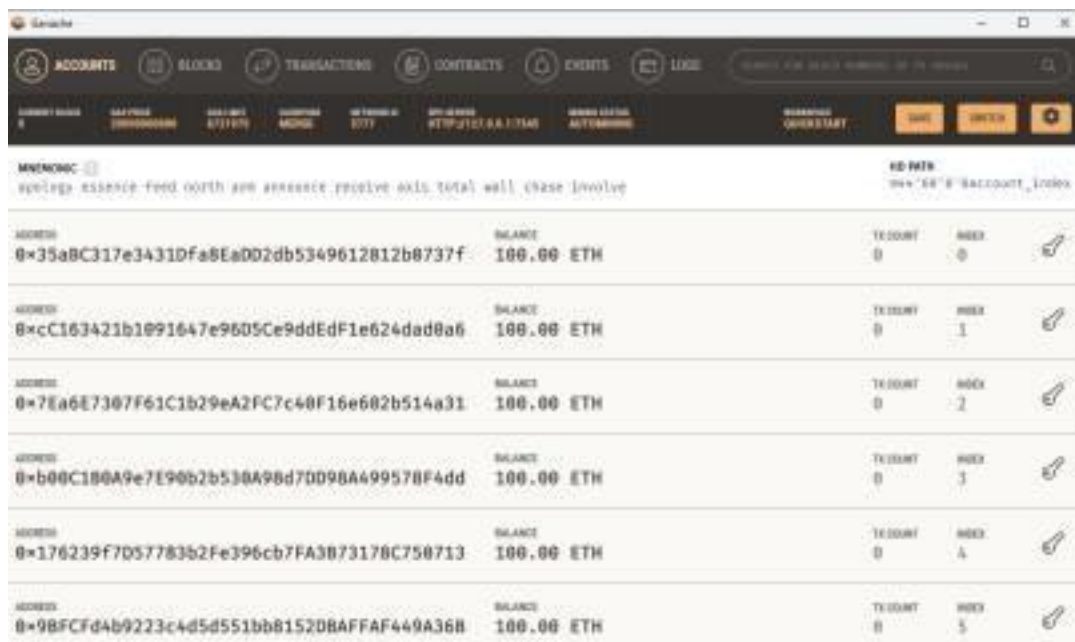
1. Download Ganache v2.7.1 from the official Truffle Suite website ([release.assets.githubusercontent.com](https://github.com/trufflesuite/ganache/releases)).

2. Run the installer. The installation prompt shows publisher as Consensys Software Inc.
3. Check 'Launch when ready' and click Install.



4. Once installed, open Ganache and click Quickstart Ethereum to start a local blockchain.





ACCOUNT INDEX	ADDRESS	BALANCE	TX COUNT	INDEX
0	0x35aBc317e3431Dfa8EaDD2db5349612812b0737f	100.00 ETH	0	0
1	0xcC163421b1091647e96D5Ce9ddEdF1e624dad8a6	100.00 ETH	0	1
2	0x7Ea6E7307F61C1b29eA2FC7c40F16e682b514a31	100.00 ETH	0	2
3	0xb00C180A9e7E90b2b530A98d7DD98A499578F4dd	100.00 ETH	0	3
4	0x176239F7D57783b2Fe396cb7FA3B73178C750713	100.00 ETH	0	4
5	0x9BFCFd4b9223c4d5d551bb8152DBAFFAF449A36B	100.00 ETH	0	5

Note the following details from Ganache for later use:

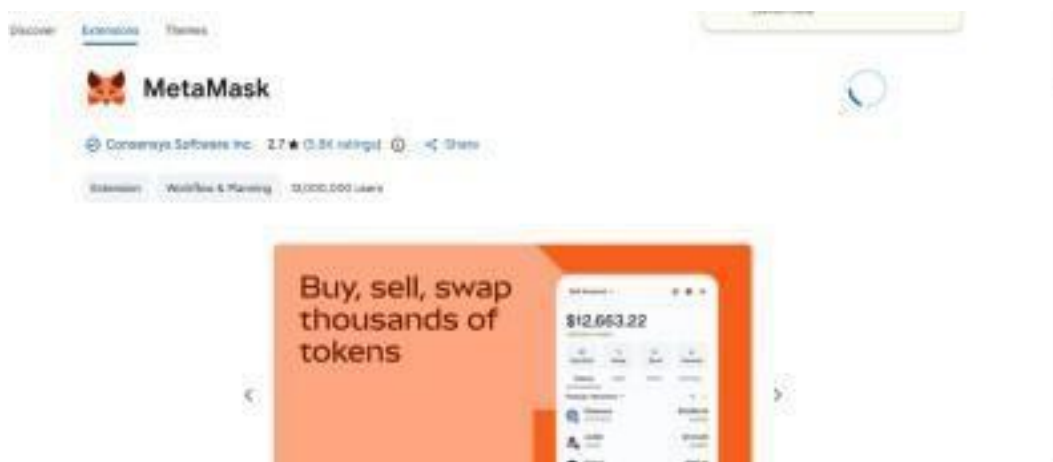
- RPC Server URL: <http://127.0.0.1:7545>
- Network ID: 5777
- Private keys of accounts (click the key icon next to each address)

Step 2: Install & Setup MetaMask

MetaMask is a browser extension that serves as an Ethereum wallet and allows interaction with decentralized applications.

2.1 Install MetaMask

5. Open Google Chrome and go to the Chrome Web Store.
6. Search for MetaMask (by Consensys Software Inc.) and click Add to Chrome.



7. After installation, click 'Create a new wallet' on the MetaMask welcome screen.

8. Set a password and save the Secret Recovery Phrase securely.



2.2 Add Ganache Network to MetaMask

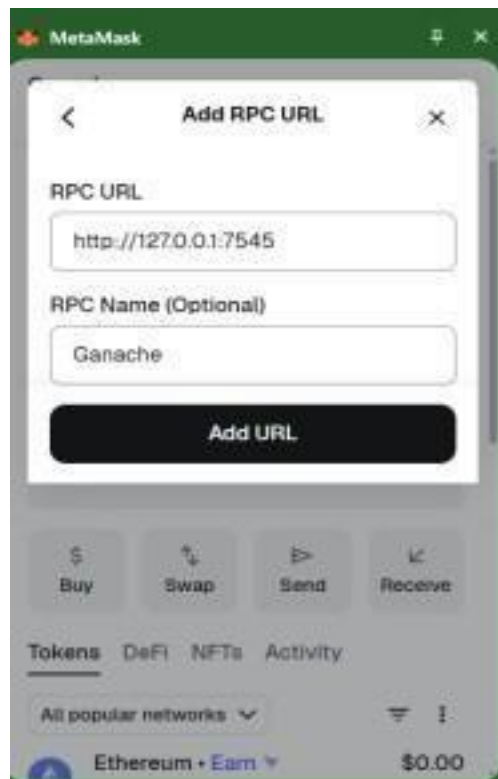
9. In MetaMask, click the network dropdown and select 'Add a custom network'.

10. Enter the following details for the RPC

URL: RPC URL: `http://127.0.0.1:7545`

RPC Name: Ganache

11. Click 'Add URL'.



12. Fill in the remaining custom network details:

- Network Name: Localhost 8545

- Default RPC URL: Ganache (127.0.0.1:7545)
- Chain ID: 1337
- Currency Symbol: ETH

13. Click Save.



The screenshot shows the 'Add a custom network' interface in MetaMask. It includes the following fields and values:

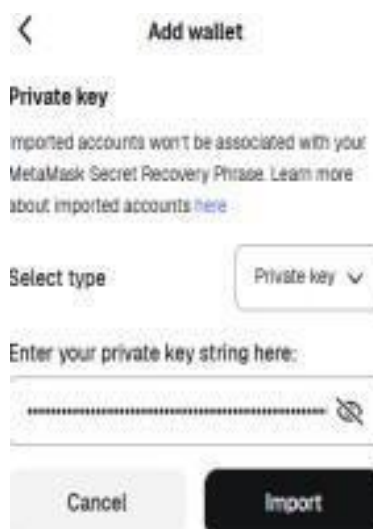
- Network name:** Localhost 8545
- Default RPC URL:** Ganache (127.0.0.1:7545)
- Chain ID:** 1337
- Currency symbol:** ETH
- Block explorer URL:** (empty)
- Action:** A black 'Save' button at the bottom.

2.3 Import Ganache Accounts into MetaMask

14. In Ganache, click the key icon next to an account address to reveal its private key.

15. In MetaMask, go to Accounts > Add wallet > Import using Private Key.

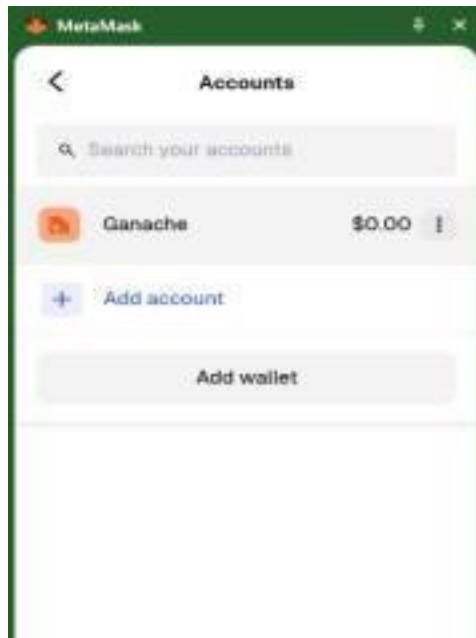
16. Paste the private key and click Import.



The screenshot shows the 'Add wallet' interface in MetaMask. It includes the following elements:

- Section Header:** Private key
- Warning:** Imported accounts won't be associated with your MetaMask Secret Recovery Phrase. [Learn more about imported accounts here](#)
- Select type:** A dropdown menu currently set to 'Private key'.
- Input Field:** A text box labeled 'Enter your private key string here:' with a masked input field containing a series of asterisks.
- Buttons:** 'Cancel' and 'Import' buttons at the bottom.

17. The imported account (labeled 'Ganache') now appears in MetaMask under the Localhost 8545 network.



Step 3: Create the Smart Contract in Remix IDE

Remix IDE is a web-based Solidity development environment at remix.ethereum.org.

18. Open remix.ethereum.org in the browser.
19. In the File Explorer, create a new file named `VotingSystem.sol`.
20. Write the following smart contract for a Voting System chain:

5777

Step 4: Compile the Smart Contract

21. In Remix, click the Solidity Compiler icon on the left sidebar.
22. Select compiler version 0.8.x.
23. Click Compile `SupplyChain.sol`.
24. A green checkmark confirms successful compilation.



Step 5: Connect Remix IDE with MetaMask

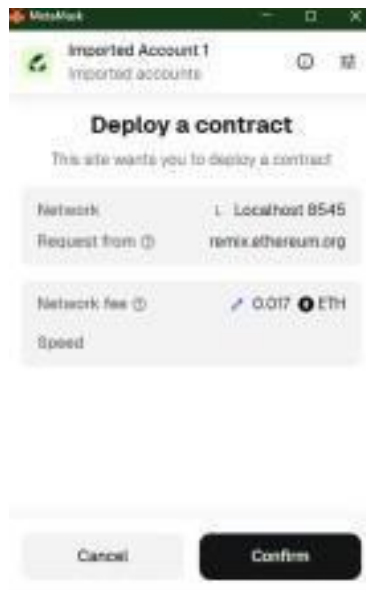
25. In Remix, navigate to the Deploy & Run Transactions tab.
26. In the Environment dropdown, select Injected Provider - MetaMask.
27. MetaMask will prompt a connection request — click Connect.
28. Ensure MetaMask is switched to the Localhost 8545 (Ganache) network.
29. The imported Ganache account will now appear as the active account in Remix.

Step 6: Deploy the Smart Contract

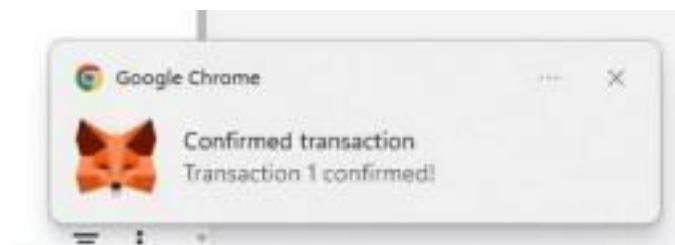
30. In the Deploy & Run panel, confirm VoteSystem is selected under Contract.
31. Click Deploy. MetaMask will open a confirmation popup.



32. Review the Network fee and click Confirm in MetaMask.



33. A 'Confirmed transaction' notification from Chrome confirms the deployment was successful.



Step 7: Verify Transaction in Ganache

After deployment, Ganache reflects the transaction. The deploying account's balance decreases due to gas fees paid.

Ganache UI Header				
ACCOUNT 1	BALANCE	GAS LIMIT	TRANSACTION	NETWORK ID
1	0.000000000000000000	0/21000	0x00	0x00
MNEMONIC: apology experts feed north arm announce receive axis total wall chase involve				
HD PATH: m/44'/0'/0'/0'				
ADDRESS	BALANCE	TX COUNT	INDEX	
0x35aBC317e3431Dfa8EaD02db5349612812b0737f	99.94 ETH	1	0	
ADDRESS	BALANCE	TX COUNT	INDEX	
0xcC163421b1091647e96D5Ce9ddEdF1e624dad8a6	100.00 ETH	0	1	

MNEMONIC: apology experts feed north arm announce receive axis total wall chase involve				
HD PATH: m/44'/0'/0'/0'				
ADDRESS	BALANCE	TX COUNT	INDEX	
0x35aBC317e3431Dfa8EaD02db5349612812b0737f	99.92 ETH	2	0	

The screenshot shows the MetaMask interface with a dark theme. At the top, there are tabs for ACCOUNTS, BLOCKS, TRANSACTIONS, CONTRACTS, EVENTS, and LOGS. Below these, there's a search bar and a list of accounts. The first account is selected, showing its address, balance (99.92 ETH), and transaction history. The transaction history shows several transactions, each with a date, amount, and status.

ADDRESS	BALANCE	TX COUNT	USED
0x35aBC317e3431Dfa8Ea002db5349612812b0737f	99.92 ETH	2	1
0xc163421b1091647e9605Ce9ddEdF1e624dad0a6	100.00 ETH	0	1
0x7Ea6E7307F61C1b29eA2FC7c40F16e602b514a31	100.00 ETH	0	2
0xb00C188A9e7E90b2b530A98d7D098A499578F4dd	100.00 ETH	0	3
0x176239f7057783b2Fe396cb7FA3B73178C750713	100.00 ETH	0	4
0x9BFCFd4b9223c4d5d551bb81520BAFFAF449A368	100.00 ETH	0	5

The contract deployment details in MetaMask show:

- Status: Confirmed
- From: 0x35aBC...0737f → To: New contract
- Gas Used: 849,377 units
- Total gas fee: 0.016988 ETH
- Account balance after: 99.92 ETH

The screenshot shows a 'Contract deployment' dialog box. It displays the status as 'Confirmed', the 'From' address as 0x35aBC...0737f, and the 'To' address as 'New contract'. Below this, a 'Transaction' section shows details like 'Amount: -0 ETH', 'Gas Limit: 849377', 'Gas Used: 849377', 'Gas fee: 0.016988 ETH', and 'Total gas fee: 0.016988 ETH'.

Contract deployment	
Status	Confirmed
From	0x35aBC...0737f
To	New contract
Transaction	
Amount	-0 ETH
Gas Limit (units)	849377
Gas Used (units)	849377
Gas fee (GWEI)	0.00252572
Priority fee (GWEI)	161367428
Total gas fee	0.016988 ETH
Max fee per gas	0.00000002 ETH
Total	0.016988 ETH

Step 8: Interact with the Smart Contract

After deployment, the contract functions appear in the Remix Deploy & Run panel under Deployed Contracts.

Transaction Flow — Pharma Supply Chain

Account 1 (Manufacturer) — Call addMedicine:

```
addMedicine(1, "Paracetamol", 100)
```

Account 2 (Distributor) — Switch account in MetaMask, then call:

```
sendToDistributor(1)
```

Account 3 (Retailer) — Switch account in MetaMask, then call:

```
sendToRetailer(1)
```

Account 4 (Customer) — Switch account in MetaMask, then call:

```
purchase(1)
```

Call getMedicine to verify the full supply chain:

```
getMedicine(1)
```

Expected output from getMedicine(1):

- id: 1
- name: Paracetamol
- quantity: 100
- manufacturer: 0x35aBC...0737f (Account 1 address)
- distributor: 0xcC163...d0a6 (Account 2 address)
- retailer: 0x7Ea6E...a31 (Account 3 address)
- customer: 0xb00C1...4dd (Account 4 address)

Output

- Medicine batch created successfully on the Ganache local blockchain.
- Complete supply chain flow executed: Manufacturer → Distributor → Retailer → Customer.
- All transactions confirmed and visible in Ganache with updated block counts and reduced ETH balances.
- Smart contract deployed with status Confirmed; gas fees deducted correctly.

Conclusion

The experiment was successfully carried out by implementing the Ganache platform and integrating it with MetaMask and Remix IDE. The smart contract was compiled, deployed, and executed using Ganache accounts, which provided a local blockchain environment with pre-funded accounts.

The transactions were processed instantly, and all details such as gas usage, block creation, and account balances were observed directly in Ganache. This provided a clear understanding of how blockchain transactions work in a controlled environment without requiring real cryptocurrency.

Thus, the experiment helped in understanding the practical implementation of a local blockchain, wallet integration, and smart contract deployment successfully.